

제구력은 하나의 숫자가 아니다

투구별 확률 예측(STEP 01)을 재료로,
상황을 보정한 투수 제구력 지표 CAE 를 만들고 검증했다 (STEP 03).

1,153.60

Private Score

+220.6

첫 제출 대비

0.751

CAE 신뢰도 (원시 0.589)

예측은 목적이 아니라 재료다

주최측 자료가 그린 흐름은 세 단계였다. 리더보드가 채점하는 것은 첫 단계뿐이지만, 질문은 마지막 단계에 있다.

단계	내용
STEP 01	투구별 제구 성공 확률 예측
STEP 02	상황별 누적
STEP 03	투수 제구력 평가

문제의 동기는 “이 투수는 정말 제구가 좋은 투수일까?” 였다. ERA·BB%·K% 는 결과만 알려준다. 그래서 우리는 예측 모델을 **지표를 만들기 위한 도구**로 쓰기로 했다.

그래서 무엇을 만들었나

같은 상황을 던진 평균 투수 대비 얼마나 더 성공시켰는가 — **CAE (Command Above Expected)**.

그리고 무엇을 증명했나

지표를 제안하는 것과 믿을 수 있음을 보이는 것은 다르다. 신뢰도·연도 간 이행·안정화 표본 셋으로 검증했다.

무엇이 잡음인지도 말한다

코치의 네 질문 중 “최근 흐름”은 측정되지 않는다. 숫자로 그렇게 나왔고, 그대로 보고한다.

결과

상위 8%, 그리고 컷 미달

숨기지 않는다. 우리 점수는 100인 컷에 23.17점 모자랐다. 이 자료의 값어치는 순위가 아니라 그 과정에서 만든 것에 있다.

1,153.60

최종 Private

79 / 990팀

팀 순위 (상위 8.0%)

240 / 1,945명

개인 순위

-23.17

100인 컷(1,176.78) 대비

기준점	점 수
주최측 공식 베이스라인 (RandomForest)	549.51
우리 첫 제출	932.96
우리 최종 (v10wtor)	1,153.60
100인 컷	1,176.78
1위	1,295.68

평가지표는 Brier Skill Score 를 10만 배 한 값이라 **상위권 전체가 1% 대의 좁은 띠 안**에 들어 있다. 베이스라인 대비 2.1배를 올렸지만 컷까지는 2.0% 가 모자랐다.

총 **48회** 제출했고, 모든 채택 판단은 리더보드 실측으로만 했다. 홀드아웃 수치로 채택한 적은 한 번도 없다 — 이유는 **5쪽**에 있다.

제출한 것 — 130피처 · 5분류 CatBoost · 30모델

단계	내용
입력	train 1,475,092행 x 49컬럼 + 트랙맨 1,793,078행 (투수 매핑 재구축본)
전처리	step1~18 파생 · 조건부 투수통계 4종 · 당해 시즌 복 원 투수 5 + 타자 2
지시자	범주형 조합 3 · 팀 관여 OR 플래그 10 · 체제 전환 1
피처	130개
타겟	5분류 — 성공 / 몰림만 / 반대만 / 둘다 / 크게벗어남
보조	교차적합 P(reverse X) 1개 (학습행 out-of-fold, 추 론행 3폴드 평균)

모델

CatBoost MultiClass (GPU) · 10-fold x 3-seed = 30 모델
iterations 1000 · depth 8 · learning_rate 0.0228
seeds [42, 202, 2024] · 학습 약 63분

후처리

폴드별 IsotonicRegression (이진 y 기준) -> 30모델 평균
-> 재중심화 로짓 상수 -0.0741
그 상수는 학습 시점 홀드아웃 (~2023 -> 2024)에서 측정해 고정한다. 평가 데이터를 보고 정하면 규
정 위반이다.

OOF 스킵 2.222% · Private 1,153.6031

학습 노트북과 제출용 script.py 는 전처리 소스 문자열 하나를 공유한다. 두 경로가 갈라지는 것이
이 프로젝트에서 가장 비쌌던 버그 유형이었다 — 트랙맨 병합 방식 불일치 하나로 36점이 났다.

상승분의 40%가 발견 하나에서 나왔다

모델 용량·하이퍼파라미터·양상블 구성은 전부 일찍 포화했다. 점수를 실제로 움직인 것은 데이터

터를 다시 읽은 다섯 번이었다.

변경	종류	리더보드
당해 시즌 성적 복원 wseason5	새 정보	+61.72
타자측 복원 wsbat	새 정보	+30.11
팀13 × 체제전환 지시자 t13	데이터 사고 진단	+20.58
타겟 5분류 multicl5	타겟 재정의	+12.85
범주형 조합 phteam	표현 비용	+7.79
반복수 1000 → 500	용량 축소	-23.07
타겟 × 구종 6분류	직교 축 분할	-82.33

검증 지표가 반복해서 틀렸다. 2024 홀드아웃은 거짓 양성(+27 → 실측 -22)도 거짓 음성(+3 → 실측 +30)도 났다. 그래서 규칙을 하나 세웠다 —

전달률은 변경의 종류에 달렸다

새 정보 ×0.85 · 버그 수정 ×0.45 · 파라미터 ×0.25 · 기존 피처 재조합 ×0.10

같은 홀드아웃 +40 이라도 어디서 온 +40 인지에 따라 리더보드 이득이 8배 차이 났다.

이 표가 다음 세 장의 이유다. 이득은 전부 “다른 행에서 정보를 끌어온” 변경에서 나왔다.

평가 행이 이미 갖고 있던 당해 시즌 정보

주최측이 준 asof_pitcher_success_rate 는 커리어 누적이라 투수마다 섞인 시즌 수가 다르다. 같은 컬럼인데 성능이 6배 갈렸다.

2024 조각	비중	그 컬럼 단독 스킵
이력 있는 투수	80.1%	141
신규 투수 (직전 시즌 없음)	19.9%	906

신규 투수는 직전 시즌이 없어 그 값이 순수한 당해 시즌 성적이다. 이력이 있으면 2019년(리그 .5647)부터 섞인 낡은 값이 된다. 즉 신선한 정보가 낡은 정보에 희석되고 있었다.

복원

커리어 누적 개수 = 비율 × asof_n (행이 갖고 있다)
- 시즌 시작 시점 누적 (train 으로 만든 투수별 록업)
= 당해 시즌 성적

규정 확인: 행 A 자신의 입력값과 공식 학습 데이터로 만든 통계만 쓴다. 평가 데이터의 다른 행을 보지 않는다. 주최측이 제시한 판정 시험 (같은 행을 단독 예측 vs 전체와 함께 예측)을 실측으로 통과했다 — 최대 차이 1.11e-16.

같은 기계를 타자측에 그대로 돌려 +30.11 을 더 얻었다. 분해 가능한 asof 컬럼 10개를 전부 확인했고, 구종 배합 3개는 0 이었다 — 성향은 이미 모델 안에 있었다.

없다고 알려진 라벨을 되살렸다

투구별 실패 유형 라벨은 제공되지 않는다. 그런데 asof_pitcher_n 이 투수 안에서 정확히 +1 씩 증가하므로, 인접 두 행의 누적 개수 차분이 곧 그 투구의 라벨이다.

검산	결과
투수 내 n 증분 +1	1.000000
복원 success vs 정답	1.000000
독립 경로(타자측)와 교차 검증	1.000000

success 는 정답 컬럼이 있어 검산이 된다. 147만 행 전부 일치했으므로 나머지 라벨도 같은 방식으로 신뢰할 수 있다. 타자측 컬럼이라는 독립 경로로 한 번 더 대조했다.

이진 타겟 하나가 5분류로 늘어난다. 성공 / 몰림만 / 반대만 / 둘다 / 크게벗어남. middle 과 reverse 는 배타적이 아니라 2비트 코드였다 (둘 다인 행이 실패의 7.2%).

유형별 드리프트가 서로 반대다 (2019→2024, 정규시즌):

유형	변화
몰림	+0.0484
반대방향	+0.0543
크게벗어남	-0.0414 반대 방향
총 변동 0.144 → 순변동	0.060 2.4배를 가린다

이진 타겟은 서로 상쇄되는 세 과정을 하나로 뭉갬다. 나누자 리더보드 +12.85. 대조군(유형 라벨만 무작위로 섞기)으로 기전도 같았다 — 분할 자체는 -23 의 비용이고 유형의 의미론이 +29 를 얻는다.

한 팀의 라벨이 시즌 중간에 다르게 매겨졌다

특정 팀이 관여한 경기의 제구 성공률이 2023년 5월에 계단처럼 바뀐다. 야구가 바뀐 게 아니라 측정계가 바뀌었다.

정규시즌, 리그 평균 대비	2022	2023	2024
해당 팀 관여 (투구의 19.9%)	-0.041	+0.051	+0.049
같은 구장을 쓰는 다른 팀	+0.001	+0.011	-0.007

대조군이 결정적이었다. 두 팀은 같은 구장을 쓴다. 하드웨어나 구장 특성이면 두 팀이 함께 움직여야 하는데 **한 팀 경기만** 움직인다.

트랙맨 원본에서도 릴리스 위치 3종과 속도 2종이 **한 달 만에 동시에** 이동했다 (extension -0.21σ). 투수는 그렇게 바뀌지 않는다.

난이도가 아니라 정의가 바뀌었다

실패 **안에서의 구성비**가 이동했다 — 반대방향 +6.5%p, 몰림 -8.0%p. 난이도 변화라면 구성비는 그대로여야 한다. ‘몰림 / 반대’의 갈림은 전적으로 **포수가 어디를 요구했**는가 추정에 달려 있다.

지시자 두 개(팀 관여, 관여 × 시점)를 수치 플래그로 주자 리더보드 **+20.58**. 6시즌을 뭉치면 부호가 상쇄되는데 **2025년은 전부 전환 후**라 그 구분이 그대로 이득이 된다.

이후 저카디널리티 컬럼 **105쌍을 전수 스캔**해 같은 유형이 더 있는지 확인했다. 하나 더 찾았고(**+7.79**), 그것으로 이 축은 닫혔다.

제출 전에 네 가지를 매번 돌렸다

주최측이 공지한 평가 데이터 독립 예측 원칙과, 학습·추론 경로가 조용히 갈라지는 것을 잡기 위한 자체 검사다. 전부 자동화돼 있다.

검사	실 측
행 독립성 — 같은 행을 단독 예측 vs 전체와 함께	1.11e-16
라벨 복원 success vs 정답 (147만 행)	1.000000
학습/추론 피처 대조 (공통 94개)	90개 완전일치
상수 생존 — 상수를 지운 판과 예측 비교	100.0%

첫 줄은 **주최측이 공지에 직접 제시한 판정 시험**이다. “어떤 행의 예측값은 test 에 그 행 하나만 있을 때와 전체가 함께 있을 때 동일해야 한다.” 그대로 구현해 매 제출 전에 돌렸다.

왜 네 번째가 필요했나

행 독립성 검사는 **규정만** 본다. 상수가 조용히 유실돼 피처가 죽어 있어도 통과한다 — 두 경우 모두 같은 값이 나오기 때문이다. 실제로 커널 산출물에서 그 버그를 잡았다. **그대로 올렸으면 61점짜리 사고였다.**

규정상 확인한 것

- 평가 데이터의 **다른 행**을 쓰지 않는다 (누적·rolling·빈도·순위 전부)
- 평가 데이터 **전체 분포**로 예측값을 보정하지 않는다
- 현재 투구 **이후**의 확정 정보를 쓰지 않는다
- 외부 데이터를 쓰지 않는다

추가로 노트북 마지막 셀이 script.py 를 **서브프로세스로 실제 실행**해 정답을 아는 시준으로 채점한다. “크래시 없이 끝났지만 점수가 0” 인 사고를 여러 번 겪고 넣은 절차다.

CAE — 상황을 건너낸 제구력

CAE = 실제 성공률

– 상황만으로 기대되는 성공률

“같은 상황을 던진 평균 투수 대비 얼마나 더 성공시켰나”. 수비 지표 OAA와 같은 형태라 현장에서 설명이 쉽다.

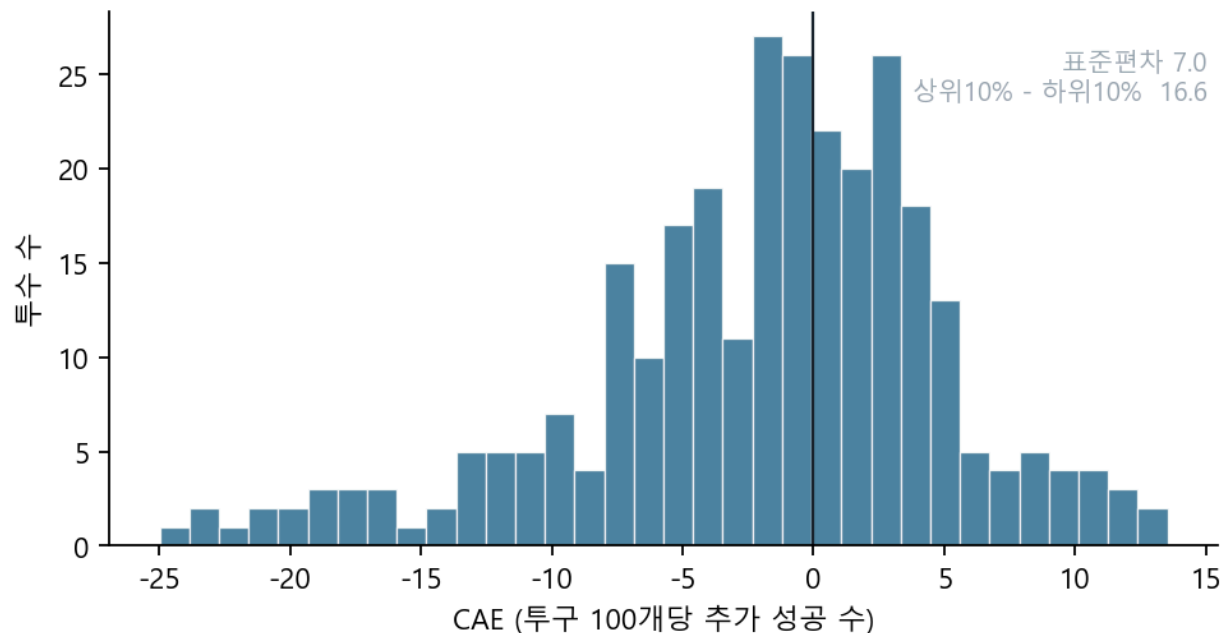
설계에서 한 번 틀렸고, 검증이 잡았다

처음엔 **대회 제출 모델의 예측**을 기대값으로 썼다. 신뢰도가 0.200으로 원시 성공률 (0.589)보다 낮게 나왔다.

당연했다 — 그 모델 입력에는 투수의 누적 통계가 들어 있어 **실력이 이미 기대값에 녹아 있다**. 빼면 잡음만 남는다.

그래서 **투수·타자 정체성과 모든 누적 통계를 제외한 상황 전용 모델**을 따로 학습했다($\sim Y-1 \rightarrow Y$). 선발과 마무리를, 4월과 9월을 같은 자에 올리는 것이 목적이다.

2024 CAE 분포 — 100구 이상 297명



2024년 100구 이상 투수 297명. 상위 10%와 하위 10%의 차이가 투구 100개당 **16.6개**다. 기대값은 홀드아웃(~ 2023 학습 $\rightarrow$ 2024 예측)에서만 만들었다 — 그 시즌을 학습에 넣으면 암기로 분산이 부풀기 때문이다.

지표를 제안하는 것과 믿을 수 있음을 보이는 것은 다르다

① 반쪽 분할 신뢰도 (2024)

값

원시 성공률	0.589
CAE (상황 보정)	0.751
(대조) 제출 모델 잔차	0.200

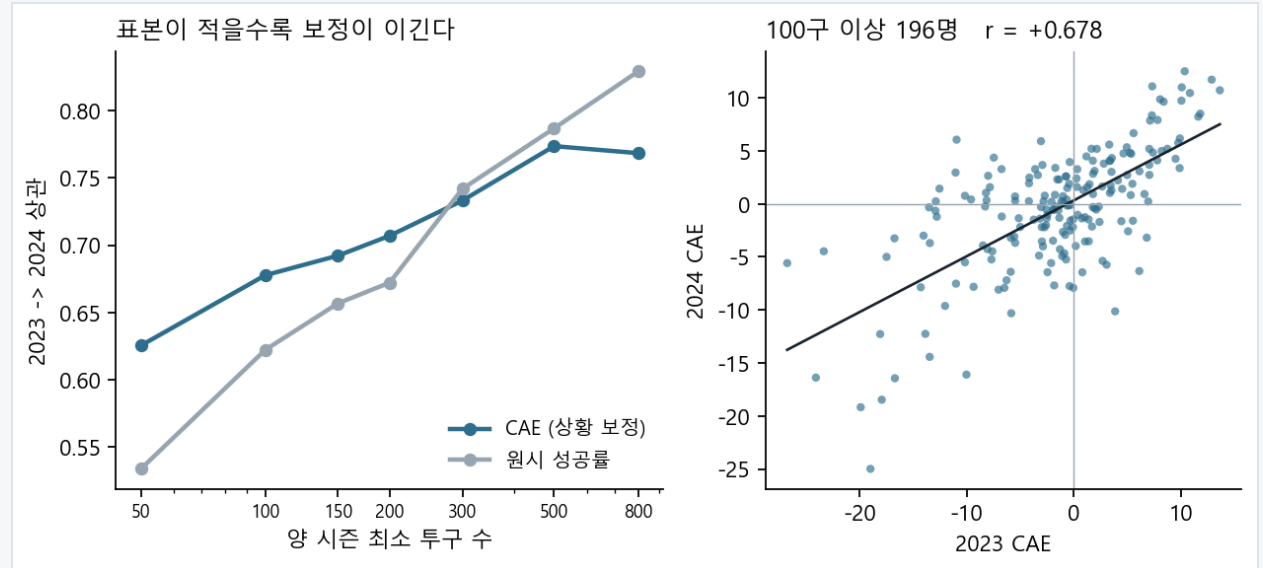
구현은 이미 아는 값으로 교정했다. 같은 코드가 기존 측정치 0.637 / -0.008 / 0.227 / -0.089 를 소수 셋째 자리까지 재현한다. **0 이 나** **와야 정상인 대조군**을 같이 두었고, 첫 구현은 거기서 +0.743 을 내며 스스로 틀렸음을 알렸다.

③ 안정화 표본 — 몇 구부터 믿나

신뢰도

20구	0.715
100구	0.817
400구	0.904

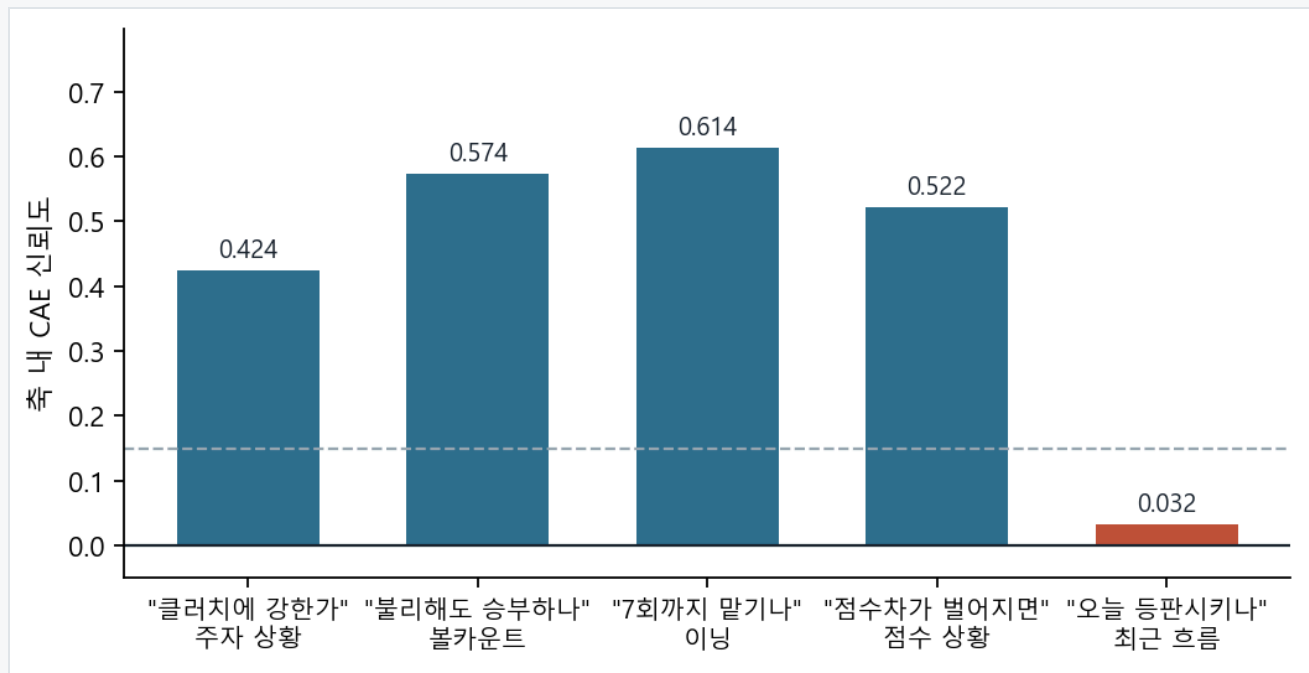
현장이 실제로 묻는 질문이다. **100구면 쓸 만하고**, 한 시즌치가 쌓이면 거의 확정이다.



② 연도 간 이행 — 선수 지표의 진짜 시험. **250구 언저리에서 교차한다**. 표본이 적으면 상황 보정이 이기고, 한 시즌치가 쌓이면 원시값이 앞선다. 원시값이 실력 외에 **보직**까지 담고 있어서다(마무리는 내년에도 마무리다). CAE 는 그 성분을 일부러 걷어낸다. 유리한 한 점만 고르지 않고 교차점을 그대로 보고한다.

코치의 질문 넷 — 셋은 답할 수 있고 하나는 잡음이다

주최측 자료가 던진 네 질문에, 각 축 안에서 CAE 가 재현되는지로 답한다. 재현되지 않으면 그 축의 차이는 실력이 아니라 표본 잡음이다.



“오늘 등판시키는 게 맞나”에는 답할 수 없다. 같은 시즌 안에서 전반과 후반의 CAE 편차는 0.032 — 사실상 0 이다. 투수 자신의 평균을 뺀 뒤 남는 변동은 재현되지 않는다.

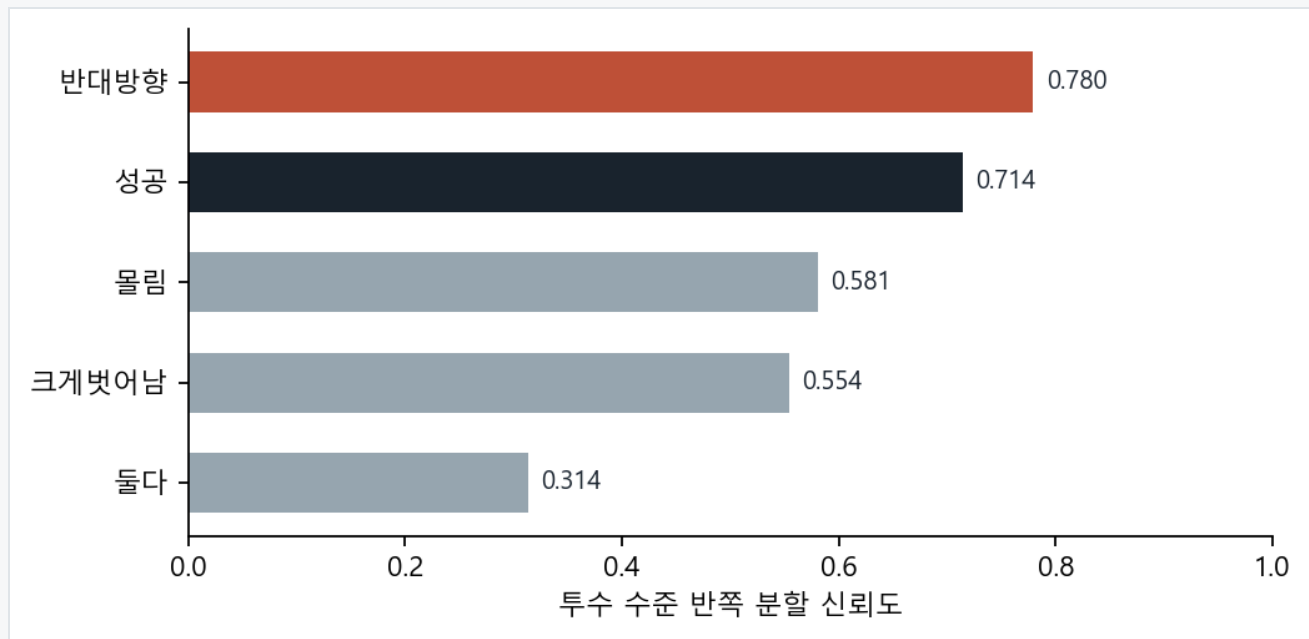
독립적인 증거가 셋이다

- 시즌 내 전후반 신뢰도 -0.089 (별도 측정)
- 최근성 가중은 리더보드에서 **5연패** (-33 / -118 / -62 / -128 / -7.95)
- 타자의 시즌별 폼 신뢰도 -0.008

이것은 실패가 아니라 결과다. 대부분의 접근은 “최근 폼” 피처를 만들고 잘 된다고 말한다. 우리는 그 축이 측정되지 않는다는 것을 서로 다른 세 방법으로 확인했고, 그래서 **거기에 시간을 쓰지 않았다.**

“얼마나” 가 아니라 “어떻게” 실패하나

복원한 라벨(7쪽)을 쓰면 하나의 성공률로는 절대 보이지 않는 것이 나온다.



“포수 요구 반대방향으로 던진다”가 전체 성공률보다 안정적인 투수
특성이다 (0.780 vs 0.714). 독립 데이터에서 같은 순서를 이미 확인
했고 (0.941 vs 0.911), CAE 시대 데이터에서 재현됐다.

현장에서 무엇이 달라지나

성공률 48% 인 두 투수라도 한쪽은 **가운데로 몰리고** 다른 쪽은 **요구
반대쪽으로 간다**. 앞은 구위·릴리스 문제이고 뒤는 사인 교환·타깃
설정 문제다. **처방이 다르다**.

같은 정보를 학습 타겟으로도 썼다. 실패를 넷으로 나눈 5분류 타겟이
리더보드 **+12.85** 였다. 즉 이 축은 **지표로도, 모델로도** 값어치가 있었
다.

남은 23점은 달을 수 있는 거리였다

우리 단일 발견 셋이 전부 그보다 컸다 (+61.72 / +30.11 / +20.58). 크기의 문제가 아니라 하나를 더 찾지 못한 문제다.

마지막 날 다섯 방향이 전부 음수였다 — 제출본은 우리가 연 축들 안에서 국소 최적이었다. 다만 그것은 우리 구성의 상한이지 문제의 상한이 아니다. **138명이 그 위에 있고, 그들이 무엇을 찾았는지 우리는 모른다.**

안 판 채로 남은 것 셋

- **신경망 블렌드** — 트리 계열끼리는 예측 상관이 0.92~0.97 에서 안 내려갔는데 임베딩 NN 만 0.67 이었다. 단독 성능을 못 올려 블렌드를 끝내지 못했다. 상관을 낮춘 유일한 후보였다.
- **트랙맨의 수준(level)** — 우리가 쓴 20개 중 18개가 표준편차다. 회전수·무브먼트·릴리스의 **값 자체**는 한 번도 안 썼다.
- **2024 홀드아웃만 보고 버린 후보 20여 개** — 그 지표가 거짓 음성을 세 번 냈으므로 (+3 → 실측 +30) 살아 있는 것이 있을 수 있다.

투수 정체성 채널 하나만 놓고 보면 (2024)

단독 스킬

주최측 커리어 누적 컬럼 그대로 43

당해 시즌 진행분 (우리가 복원한 것) 600

당해 시즌 **전체** (이 투구 이후까지) 724

마지막 줄은 “이 투구 이후의 기록”이라 평가 시점에 존재하지 않는다. **이 표가 말하는 것은 이 채널이 닫혔다는 것뿐이다** — 다른 채널까지 닫혔다는 뜻이 아니다. 우리가 회수한 구간은 43 → 600 이다.

지표 쪽에 남은 재료

- **포수 ID** 가 있으면 CAE 를 배터리 단위로 분해할 수 있다. 라벨 자체가 “포수 요구 지점”에 달려 있다.
- **홈플레이트 통과 위치** 가 있으면 실패의 **크기**를 잴 수 있다. 지금은 유형까지만 구분된다.